

Student: Farukh Kharov | Student ID: 202444907

Classification of Household Waste Using Transfer Learning

Mini-project for the Deep Learning course

Image classification (영상 데이터 분류) · 10 classes

1. Problem Statement

This project solves an **image classification** problem: automatic sorting of household waste into categories for a recycling system. This is one of the three proposed assignment topics, specifically image classification.

Parameter	Value
Task type	Multiclass image classification
Input variable	RGB image resized to 224×224×3 pixels
Output variable	One of 10 classes: battery, biological, cardboard, clothes, glass, metal, paper, plastic, shoes, trash
Metrics	Accuracy and macro F1

The practical significance of this task is the automation of waste separation, which belongs to relevant environmental ESG directions. The main method selected for this project is **Transfer Learning**, which transfers features from a model pretrained on ImageNet. This is an industry standard when the amount of data is limited.

2. Dataset Description

The external dataset **Garbage Classification V2** was used. The dataset was created by Suman Kunwar and published on Kaggle. The dataset is described in the scientific article “The Garbage Dataset (GD): A Multi-Class Image Benchmark for Automated Waste Segregation” (2026), where the author reports a benchmark of 95.13% accuracy for the EfficientNetV2S architecture.

Characteristic	Value
Total number of images	12,259
Number of classes	10
Format	JPEG, RGB, 256×256 size (standardized_256 version)
train / val / test split	70% / 15% / 15%, stratified
train / val / test examples	8593 / 1833 / 1833

Class Distribution and Imbalance

The dataset is significantly imbalanced: the ratio between the largest and smallest class is **4.18×** (clothes: 1892, trash: 453). This characteristic directly influenced the choice of preprocessing methods, especially class weighting.

Class	Count	Share
clothes	1892	15.4%
glass	1736	14.2%
plastic	1597	13.0%
shoes	1449	11.8%
cardboard	1411	11.5%
paper	1336	10.9%
metal	930	7.6%
battery	756	6.2%
biological	699	5.7%
trash	453	3.7%

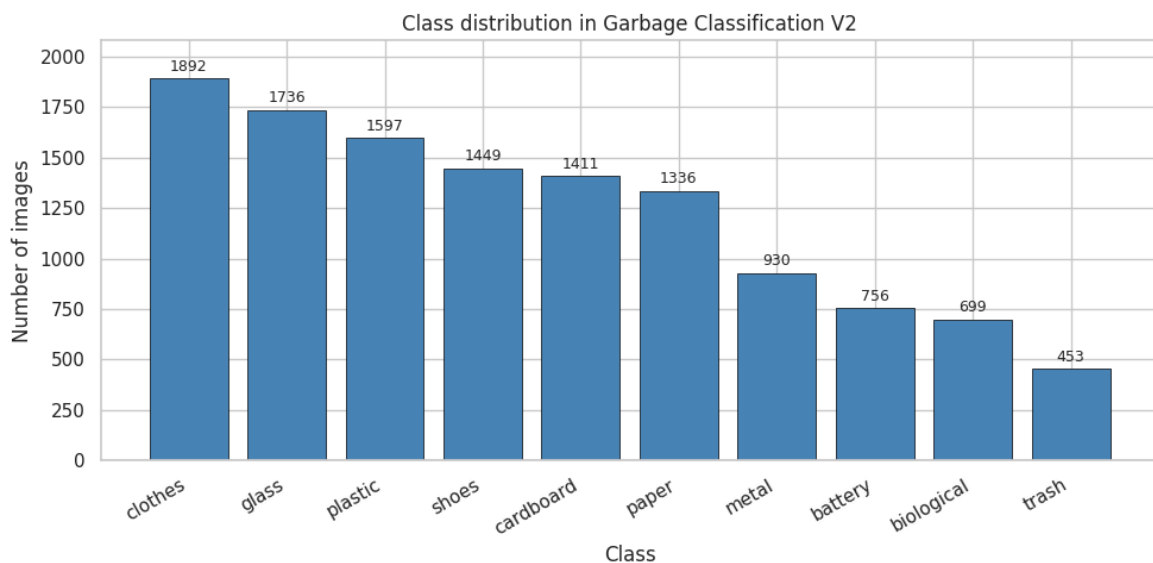


Figure 1. Distribution of images by class

Exploratory Data Analysis (EDA) Results

A visual inspection of image samples from each class showed the following:

- **High intra-class variability.** The classes battery (different types of batteries), biological (bread, vegetables, fruit, pizza), trash (masks, gloves, packaging), and paper (newspapers, sheets, receipts) are visually very diverse.
- **Predicted confusion pairs.** Based on visual similarity, three pairs were predicted in advance as likely to be confused by the model: paper ↔ cardboard, glass ↔ metal, plastic ↔ glass. All three were confirmed during the experiments.
- **Artifacts.** Some images contain gray border areas, which are a result of resizing to 256×256. This justified the use of random crop as an augmentation method.
- **Technical consistency.** All checked images were confirmed to be 256×256, RGB, JPEG.

Sample images from each class

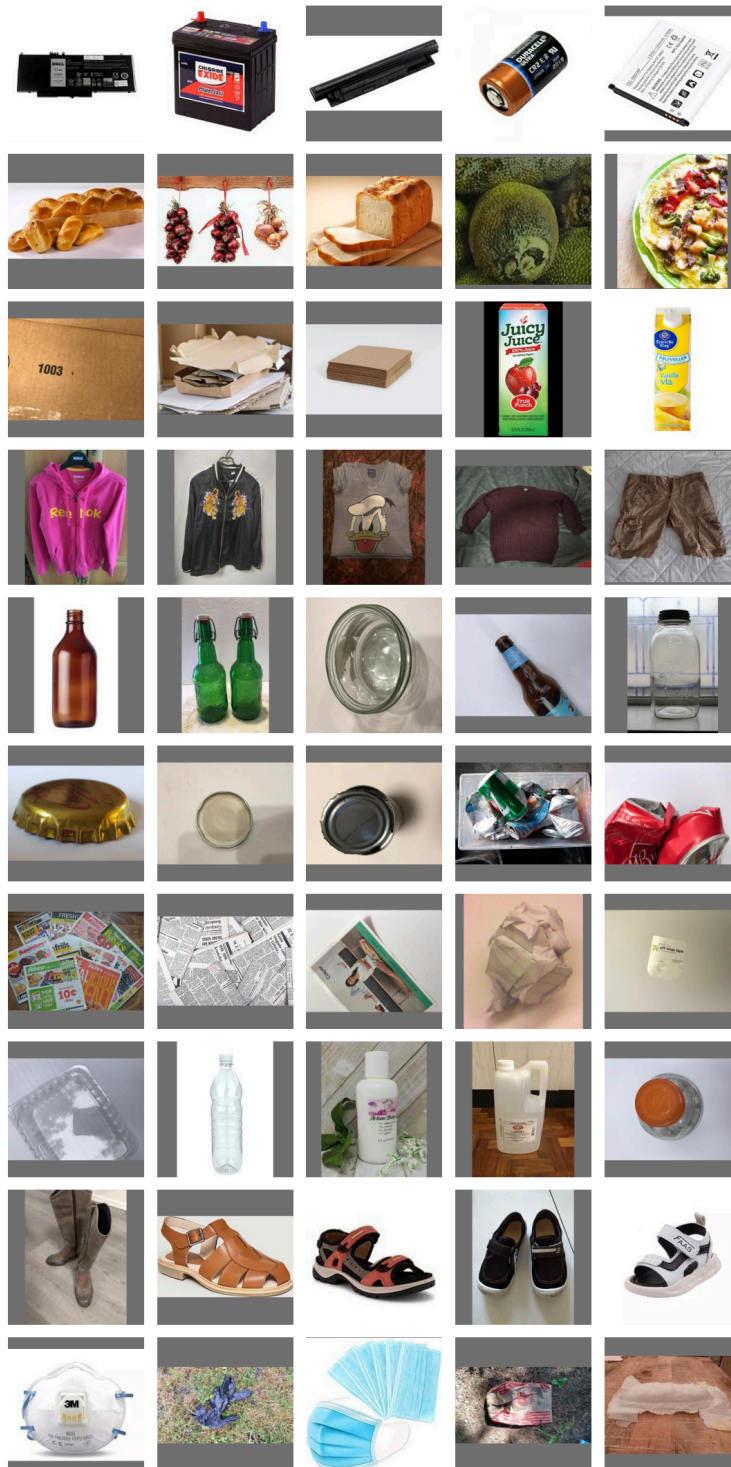


Figure 2. Example images from each class

3. Data Preprocessing

A custom data processing pipeline (`src/data.py`) was implemented based on `tf.data` :

Stage	Description
Stratified split	Each class is split independently in the 70/15/15 ratio, preserving class proportions in all subsets with an error of less than 0.1%
Augmentation (train only)	Random crop to 224×224, horizontal flip, brightness and contrast adjustment ($\pm 15\%$)
val / test processing	Deterministic center crop to 224×224 without augmentations, for metric reproducibility
Class weighting	Weights based on inverse frequency: trash 2.71×, biological 1.75×, battery 1.62×, clothes 0.65×
Pipeline optimization	Parallel loading, shuffling, and prefetch for efficient GPU usage

Stratification was verified empirically: the deviation of class proportions between train/val/test did not exceed 0.05%, which is critical for reliable evaluation on an imbalanced dataset.

4. Hyperparameter Optimization and Experiments

A total of **five experiments** were conducted, while the minimum assignment requirement was three. All experiments used the same training pipeline (`src/train.py`) with `EarlyStopping`, `ReduceLROnPlateau`, and `ModelCheckpoint` callbacks, which ensured a fair comparison. Training was performed on Kaggle using a GPU P100.

No.	Model / Configuration	Test Acc	Macro F1	Epochs	Time
1	Baseline CNN (from scratch, 4 convolutional blocks)	78.61%	0.780	30	5.4 min
2	ResNet50, frozen base	92.69%	0.920	17	4.5 min
3	ResNet50, fine-tune	93.29%	0.925	26	7.1 min
4	EfficientNetV2S, frozen (224px)	92.53%	0.919	20	8.2 min
5	ResNet50 fine-tune + class weights	93.51%	0.930	28	7.7 min

The best configuration is highlighted in yellow.

Experiment 1 — Baseline CNN

A simple convolutional network was trained from scratch. It consisted of 4 Conv→BatchNorm→ReLU→MaxPool blocks and had approximately 390K parameters. The achieved accuracy was **78.61%**. An important observation is that train accuracy (77.7%) was close to validation accuracy (78.9%), meaning there was no overfitting. The augmentations worked as an effective regularizer. The weakest class was trash (F1 = 0.655), while the strongest classes were clothes (0.880), biological (0.874), and battery (0.852). All three confusion pairs predicted during EDA were confirmed.

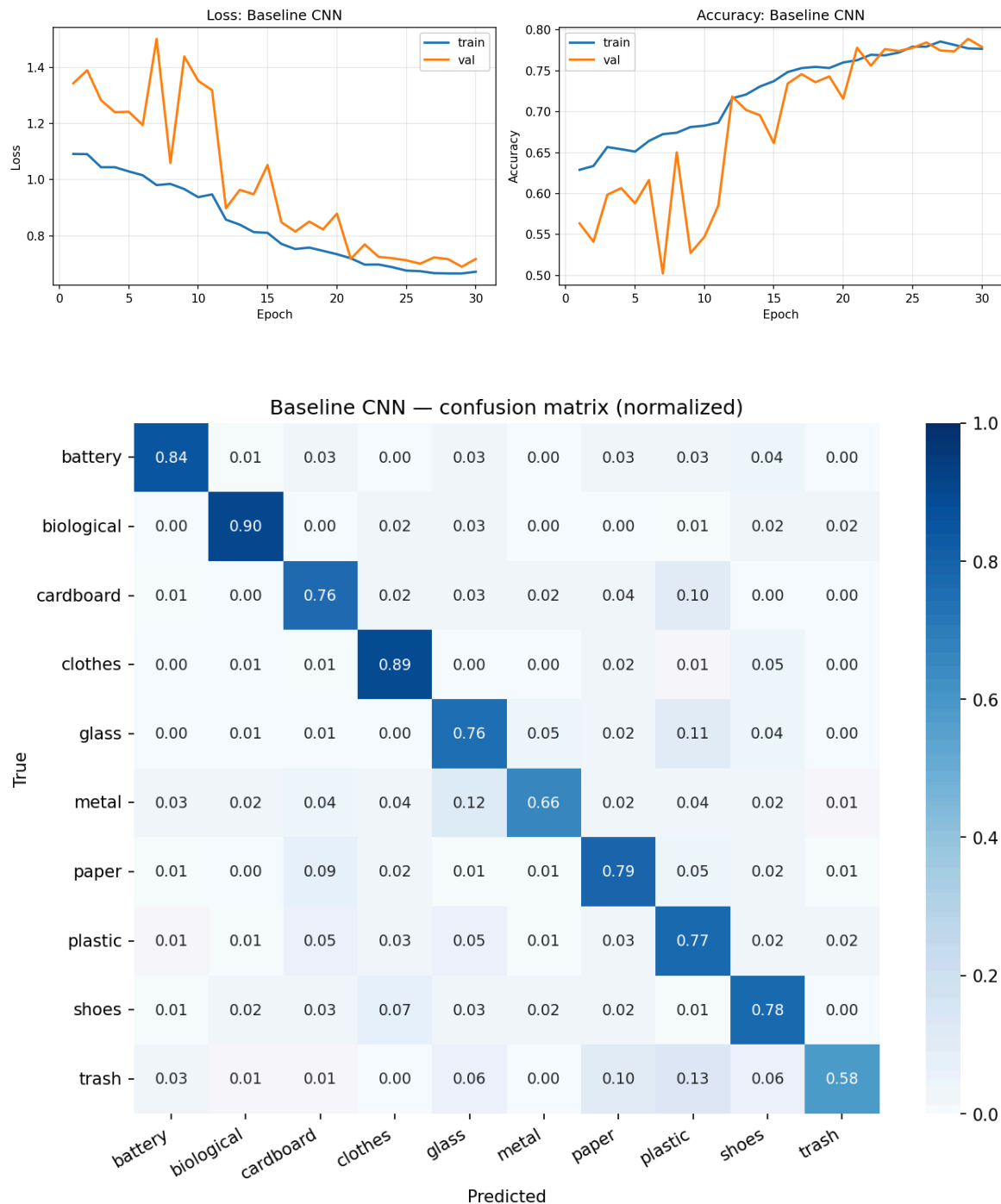


Figure 3. Learning curves and confusion matrix of Baseline CNN

Experiment 2 — ResNet50 (Frozen Base)

Transfer Learning was applied: ResNet50 with ImageNet weights was used as a frozen feature extractor, and only the classifier was trained. Only about 20K parameters were trained out of 23.6M total parameters. The result was **92.69%**, an improvement of **+14.08 percentage points** over the baseline. Already in the first epoch, validation accuracy reached 89.7%, which was higher than the final result of the baseline. All classes improved, with the largest gains in previously weak classes: metal (+0.24 recall), trash (+0.23), and shoes (+0.22).

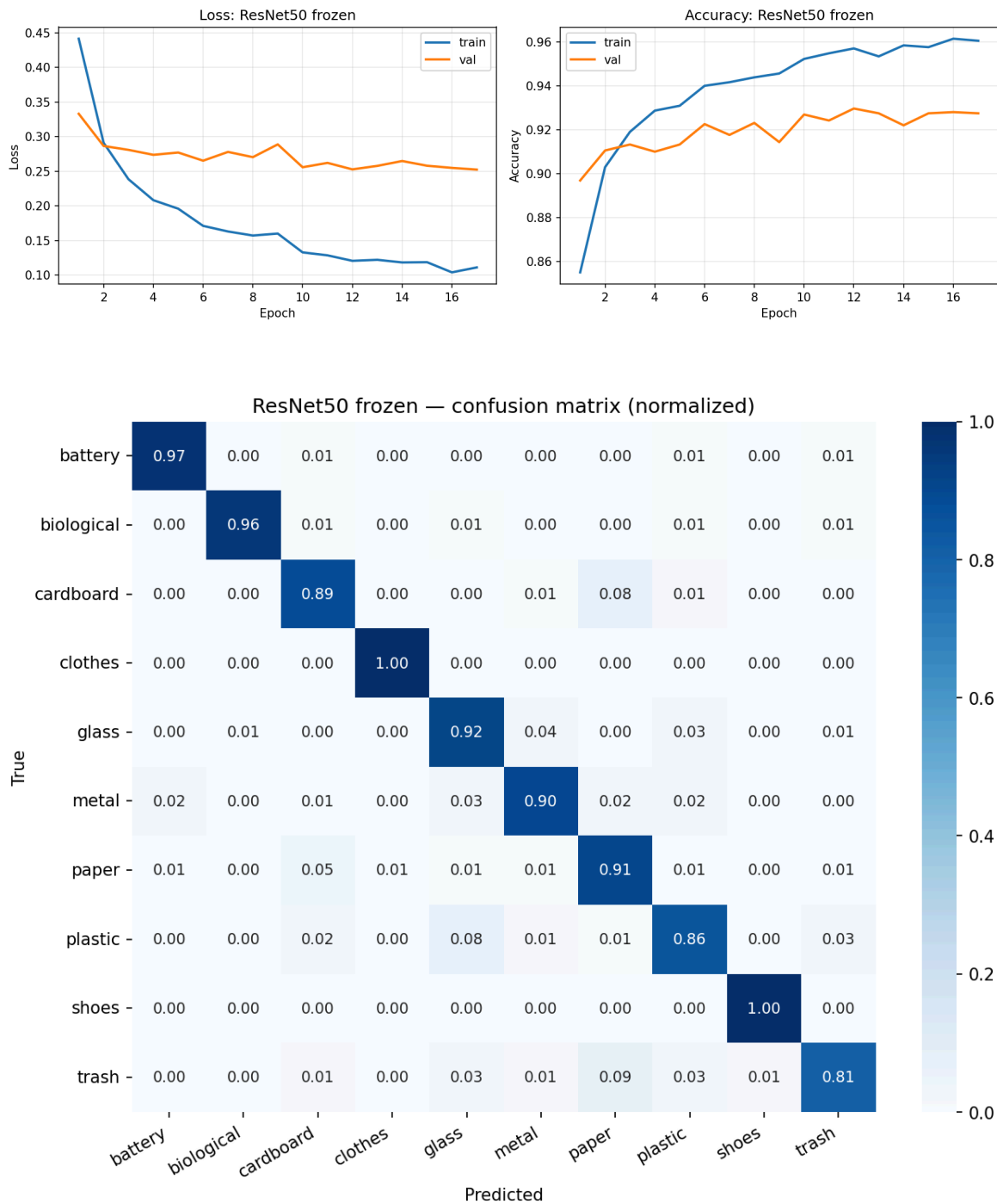


Figure 4. Learning curves and confusion matrix of ResNet50 frozen

Experiment 3 — ResNet50 (Fine-tuning)

Two-phase training was used. Phase 1 consisted of training the classifier while keeping the base frozen. In phase 2, the last ResNet50 block (conv5_block3) was unfrozen and fine-tuned with a learning rate reduced by 100 times ($1e-5$). BatchNorm layers remained frozen to preserve ImageNet statistics. The result was **93.29%**. The main achievement was that plastic→glass confusion decreased from 8% to 5%, which is a relative improvement of 38%. This confirms the hypothesis that fine-tuning is useful for visually similar classes.

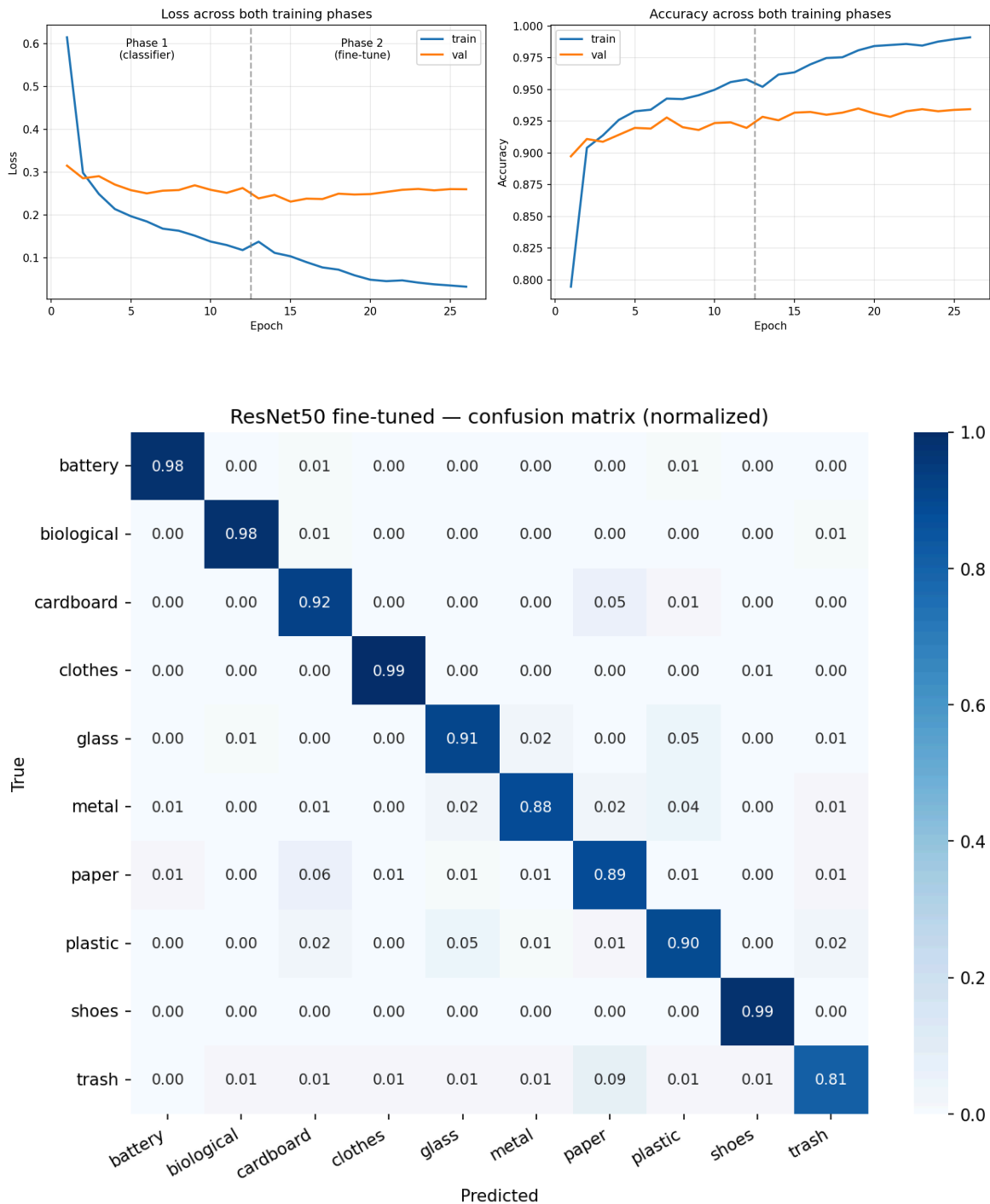


Figure 5. Learning curves with two phases and confusion matrix of ResNet50 fine-tuned

Experiment 4 — EfficientNetV2S

An alternative and more modern architecture was compared. The result was **92.53%**, which was slightly lower than ResNet50. This is an important methodological conclusion: EfficientNetV2S was designed for 384×384 input based on the principle of compound scaling, while this project used 224×224 for a fair comparison with ResNet50. This explains both the lower result compared to ResNet50 and the 2.6 percentage point gap from the article benchmark of 95.13% at 384×384. At the same time, EfficientNetV2S showed the lowest overfitting, with a train-val gap of only about 1.5%.

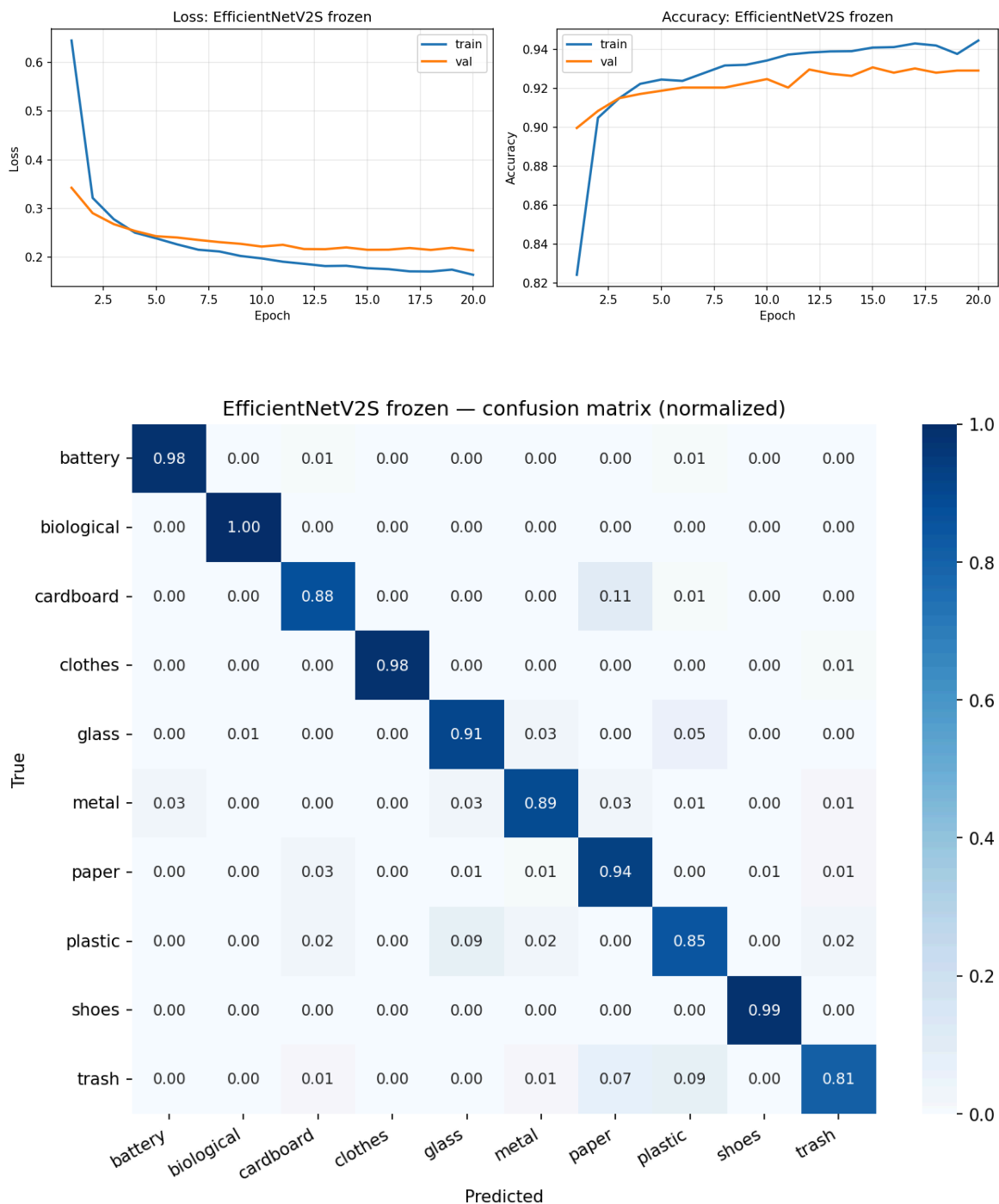


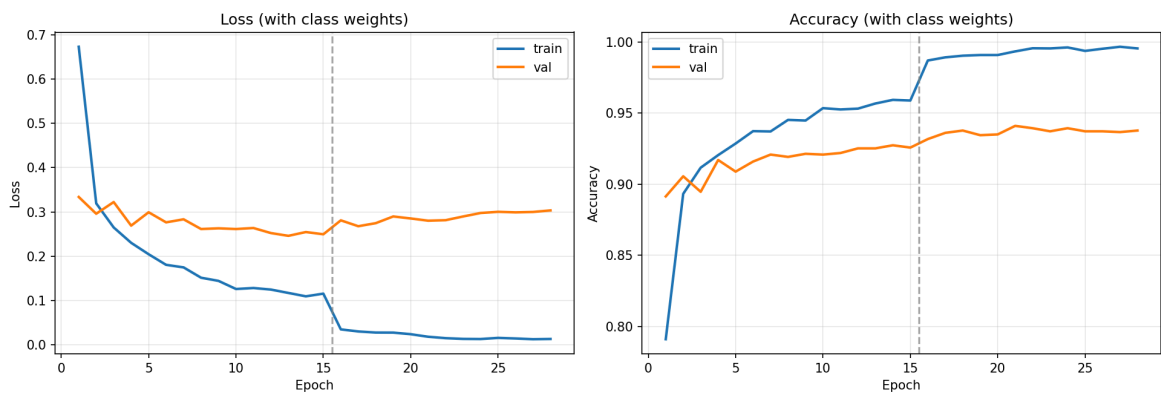
Figure 6. Learning curves and confusion matrix of EfficientNetV2S

Experiment 5 — ResNet50 Fine-tune + Class Weights (Best Model)

Class weights based on inverse frequency were added to the best model from Experiment 3. The result was **93.51%**, the best result in the project. The unique point is that both overall accuracy and macro F1 improved at the same time. Class weighting often sacrifices overall accuracy, but in this case moderate weights improved performance in all directions.

The key achievement was breaking the “ceiling” of the trash class. In Experiments 2-4, the recall of the trash class was stuck at around 0.81 regardless of architecture. Class weighting increased it to **0.881** (+7.5 percentage points). This confirmed the diagnosis made during EDA: the limiting factor for trash was data imbalance, not feature quality or architecture.

Trash class	Without weights	With weights	Δ
recall	0.806	0.881	+0.075
precision	0.806	0.787	−0.019
F1	0.806	0.831	+0.025



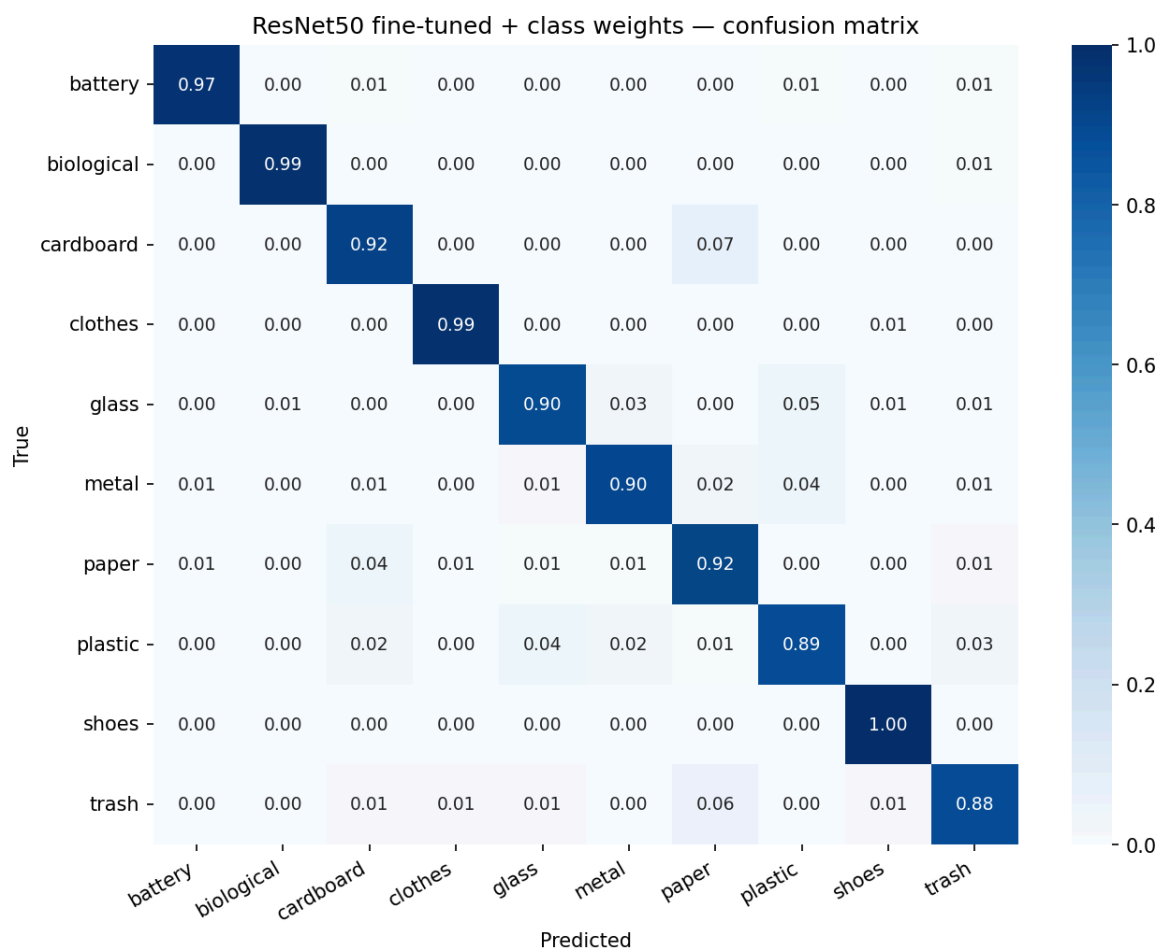


Figure 7. Learning curves and confusion matrix of the best model

5. Results

Final Hyperparameters (Best Model)

Hyperparameter	Value
Architecture	ResNet50 (ImageNet) + GlobalAveragePooling + Dropout(0.3) + Dense(10)
Training strategy	Two-phase: classifier → fine-tuning of the last block
Optimizer	Adam
Learning rate	1e-3 (phase 1) → 1e-5 (phase 2), with ReduceLROnPlateau
Batch size	32
Loss function	Sparse Categorical Crossentropy (from logits)
Regularization	Augmentation, Dropout, EarlyStopping
Class weighting	Inverse frequency (trash 2.71× ... clothes 0.65×)
Input size	224×224×3

Final Performance

Metric	Value
Test accuracy	93.51%
Test macro F1	0.930
Best class (clothes, F1)	0.989
Worst class (trash, F1)	0.831
Improvement over baseline	+14.90 percentage points
Gap from the article benchmark (95.13%, 384px)	−1.62 percentage points

Conclusions

- **Transfer Learning made the decisive contribution.** Moving from training from scratch (78.6%) to ResNet50 (92.7%) added more than 14 percentage points by transferring semantic features from ImageNet.
- **Fine-tuning and class weighting** increased the result to 93.5% and solved the problem of the rare trash class diagnosed during EDA.
- **Architecture cannot be separated from input resolution.** EfficientNetV2S did not outperform ResNet50 at 224px, which is consistent with the theory of compound scaling.
- **The value of EDA was confirmed.** The confusion pairs predicted from visual analysis (paper/cardboard, glass/metal, plastic/glass) were confirmed in the confusion matrices, and the diagnosis of the trash problem as an imbalance issue rather than a feature issue was confirmed through a series of experiments.

Possible Future Improvements

- Use the 384×384 version of the dataset with EfficientNetV2S to approach the 95.13% benchmark.
- Test generalization ability on an independent dataset, such as TrashNet, using an Out-of-Distribution test.

Code and reproducibility. All code is organized as reusable modules (`src/data.py` , `model.py` , `train.py` , `utils.py`) and experiment notebooks.

GitHub: <https://github.com/wayofell/trash-classification>